Natural Language Processing

Complete Concepts, Sub-Concepts & Python Implementations

A structured reference covering the full NLP pipeline — from text preprocessing to transformers — with runnable Python code for every concept.

Contents

1. Introduction to NLP

Natural Language Processing (NLP) is the branch of Artificial Intelligence that gives machines the ability to read, understand, interpret, and generate human language, in both text and speech form. It sits at the intersection of linguistics, computer science, and machine learning.

1.1 Why NLP is Hard

- **Ambiguity** — the same word/sentence can have multiple meanings (lexical, syntactic, semantic).
- **Context dependence** — meaning changes based on surrounding words, tone, and world knowledge.
- **Variability** — slang, dialects, spelling errors, code-mixing, and evolving vocabulary.
- **Structure** — languages differ hugely in grammar, morphology, and word order.

1.2 Applications of NLP

- Search engines & information retrieval
- Machine translation (Google Translate)
- Chatbots & virtual assistants (Siri, Alexa)
- Sentiment & opinion analysis
- Text summarization
- Spam & fraud detection
- Speech recognition & synthesis
- Large Language Models (ChatGPT, Claude, Gemini)

1.3 Levels of Language Analysis

- **Phonology** — sound structure of language
- **Morphology** — structure of words (roots, prefixes, suffixes)
- **Syntax** — grammatical structure of sentences
- **Semantics** — meaning of words and sentences
- **Pragmatics** — meaning in context/discourse
- **Discourse** — meaning across multiple sentences

1.4 Setting Up Your Environment

```
# Core NLP libraries used throughout this guide
pip install nltk spacy scikit-learn gensim transformers torch rouge-score

python -m spacy download en_core_web_sm

import nltk
nltk.download('punkt')
```

```
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('maxent_ne_chunker')
nltk.download('words')
```

2. The NLP Pipeline

Most NLP systems follow a common pipeline. Understanding each stage helps in debugging and designing new systems.

- **1. Data Acquisition** — collect raw text (web scraping, APIs, corpora)
- **2. Text Preprocessing** — cleaning, tokenization, normalization
- **3. Feature Engineering / Representation** — BoW, TF-IDF, embeddings
- **4. Modeling** — statistical or neural models (classification, sequence labeling, generation)
- **5. Evaluation** — accuracy, F1, BLEU, ROUGE, perplexity
- **6. Deployment** — serving the model via APIs, apps, etc.

Note: Modern LLM-based systems (e.g., using pretrained Transformers) often compress steps 3–4 into a single pretrained model that is fine-tuned or prompted directly.

3. Text Preprocessing

Raw text is noisy. Preprocessing converts it into a clean, consistent form suitable for feature extraction and modeling.

3.1 Tokenization

Splitting text into smaller units — words, subwords, or sentences.

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Dr. Smith went to Washington. He loves NLP!"

print(sent_tokenize(text))
# ['Dr. Smith went to Washington.', 'He loves NLP!']

print(word_tokenize(text))
# ['Dr.', 'Smith', 'went', 'to', 'Washington', '.', 'He', 'loves', 'NLP', '!']

# Using spaCy (handles punctuation/edge cases robustly)
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)
print([token.text for token in doc])
```

3.1.1 Subword Tokenization (BPE / WordPiece)

Used by modern Transformer models (BERT, GPT) to handle rare/unseen words by breaking them into frequent sub-units.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer.tokenize("unbelievably fascinating")
print(tokens)
# ['unbelievably', 'fascinating']  (or split into sub-tokens for rare words)

ids = tokenizer.encode("unbelievably fascinating")
print(ids)
```

3.2 Stopword Removal

Removing very common words (the, is, at, on) that carry little semantic weight for many tasks.

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
words = word_tokenize("This is an example showing stopword removal")
filtered = [w for w in words if w.lower() not in stop_words]
print(filtered)
# ['example', 'showing', 'stopword', 'removal']
```

3.3 Stemming

Reduces words to a crude root form by chopping suffixes (fast, but not always a real word).

```
from nltk.stem import PorterStemmer, SnowballStemmer
```

```
porter = PorterStemmer()
snowball = SnowballStemmer("english")

words = ["running", "flies", "easily", "studies"]
print([porter.stem(w) for w in words])
# ['run', 'fli', 'easili', 'studi']
print([snowball.stem(w) for w in words])
```

3.4 Lemmatization

Reduces words to their dictionary (base) form using vocabulary and morphological analysis — more accurate than stemming.

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

print(lemmatizer.lemmatize("running", pos="v"))   # run
print(lemmatizer.lemmatize("studies", pos="n"))   # study
print(lemmatizer.lemmatize("better", pos="a"))    # good

# spaCy lemmatization (automatic POS-aware)
doc = nlp("The striped bats were hanging on their feet")
print([token.lemma_ for token in doc])
```

3.5 Text Normalization

Standardizing text: lowercasing, removing punctuation/numbers, expanding contractions, handling special characters.

```
import re

def normalize(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "", text)      # remove URLs
    text = re.sub(r"[^a-z\s]", "", text)             # keep only letters
    text = re.sub(r"\s+", " ", text).strip()         # collapse whitespace
    return text

print(normalize("Check this out: https://x.com!! Amazing #NLP 2024"))
# "check this out amazing nlp"
```

4. Text Representation

Machine learning models need numbers, not raw text. This section covers ways to turn text into numeric vectors — from simple counts to deep contextual embeddings.

4.1 Bag of Words (BoW)

Represents a document as an unordered set of word counts, ignoring grammar and order.

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["I love NLP", "NLP is fun", "I love machine learning"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

4.2 TF-IDF (Term Frequency - Inverse Document Frequency)

Weighs words by how important they are to a document relative to the whole corpus — common words get down-weighted.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(corpus)

print(tfidf.get_feature_names_out())
print(X_tfidf.toarray().round(2))
```

4.3 N-grams

Contiguous sequences of N tokens; captures local word order (bigrams, trigrams).

```
from sklearn.feature_extraction.text import CountVectorizer

bigram_vectorizer = CountVectorizer(ngram_range=(2, 2))
X_bigrams = bigram_vectorizer.fit_transform(["I love natural language processing"])
print(bigram_vectorizer.get_feature_names_out())
# ['language processing' 'love natural' 'natural language']
```

4.4 Word Embeddings

Dense, low-dimensional vectors that capture semantic similarity — words with similar meaning end up close in vector space.

4.4.1 Word2Vec (CBOW & Skip-gram)

```
from gensim.models import Word2Vec

sentences = [["i", "love", "nlp"], ["nlp", "is", "fun"], ["i", "love", "machine", "learning"]]

model = Word2Vec(sentences, vector_size=50, window=3, min_count=1, sg=1)  # sg=1 -> skip-gram
print(model.wv["nlp"])                # embedding vector for 'nlp'
print(model.wv.most_similar("love"))  # nearest neighbours
```

4.4.2 Pretrained GloVe Embeddings

```
import gensim.downloader as api

glove = api.load("glove-wiki-gigaword-50")   # downloads pretrained vectors
print(glove["king"] - glove["man"] + glove["woman"])
print(glove.most_similar(positive=["king", "woman"], negative=["man"]))
# -> ['queen', ...]
```

4.4.3 FastText (Subword-aware Embeddings)

```
from gensim.models import FastText

ft_model = FastText(sentences, vector_size=50, window=3, min_count=1)
print(ft_model.wv["learning"])
# Handles out-of-vocabulary words via character n-grams
print(ft_model.wv["learnings"])
```

4.5 Contextual Embeddings (BERT)

Unlike Word2Vec/GloVe, contextual embeddings produce a different vector for a word depending on its surrounding sentence.

```
from transformers import AutoTokenizer, AutoModel
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

text = "The bank raised interest rates."
inputs = tokenizer(text, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)

embeddings = outputs.last_hidden_state   # shape: [1, seq_len, 768]
print(embeddings.shape)
```

5. Syntactic Analysis

Syntax analysis examines the grammatical structure of sentences.

5.1 Part-of-Speech (POS) Tagging

Assigns a grammatical category (noun, verb, adjective...) to each token.

```
import nltk
tokens = word_tokenize("The quick brown fox jumps over the lazy dog")
print(nltk.pos_tag(tokens))
# [('The','DT'), ('quick','JJ'), ('brown','JJ'), ('fox','NN'), ('jumps','VBZ') ...]

# spaCy version (with human-readable tags)
doc = nlp("The quick brown fox jumps over the lazy dog")
for token in doc:
    print(token.text, token.pos_, token.tag_)
```

5.2 Constituency Parsing

Breaks a sentence into nested sub-phrases (noun phrases, verb phrases) forming a tree, based on a phrase-structure grammar.

```
import nltk
grammar = nltk.CFG.fromstring('''
  S -> NP VP
  NP -> Det N
  VP -> V NP
  Det -> 'the'
  N -> 'dog' | 'cat'
  V -> 'chased'
''')
parser = nltk.ChartParser(grammar)
for tree in parser.parse(['the', 'dog', 'chased', 'the', 'cat']):
    tree.pretty_print()
```

5.3 Dependency Parsing

Models grammatical relationships between words as directed edges (head -> dependent), rather than nested phrases.

```
doc = nlp("She sells sea shells on the sea shore")
for token in doc:
    print(f"{token.text:10} --{token.dep_:10}--> {token.head.text}")

# Visualize (in Jupyter):
# from spacy import displacy
# displacy.render(doc, style="dep")
```

5.4 Chunking (Shallow Parsing)

Groups tokens into flat, non-recursive phrases like noun phrases (NP) or verb phrases (VP).

```
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunk_parser = nltk.RegexpParser(grammar)
tagged = nltk.pos_tag(word_tokenize("The quick brown fox jumps"))
```

```
tree = chunk_parser.parse(tagged)
print(tree)
```

6. Semantic Analysis

Semantic analysis focuses on extracting meaning from text.

6.1 Named Entity Recognition (NER)

Identifies and classifies real-world entities: people, organizations, locations, dates, etc.

```
doc = nlp("Apple was founded by Steve Jobs in Cupertino in 1976.")
for ent in doc.ents:
    print(ent.text, ent.label_)
# Apple ORG | Steve Jobs PERSON | Cupertino GPE | 1976 DATE
```

6.2 Word Sense Disambiguation (WSD)

Determines which meaning of an ambiguous word is used in a given context (e.g., 'bank' = river bank vs. financial bank).

```
from nltk.wsd import lesk
sentence = word_tokenize("I went to the bank to deposit money")
sense = lesk(sentence, 'bank')
print(sense, "->", sense.definition())
```

6.3 Semantic Role Labeling (SRL)

Identifies 'who did what to whom, where, and when' by labeling predicate-argument structures (e.g., using AllenNLP's SRL model).

```
# Example using allennlp (conceptual - requires allennlp-models)
# from allennlp.predictors.predictor import Predictor
# predictor = Predictor.from_path(
#     "https://storage.googleapis.com/allennlp-public-models/structured-prediction-srl-bert.tar.gz")
# result = predictor.predict(sentence="The dog chased the cat across the yard.")
# print(result['verbs'])
```

6.4 Coreference Resolution

Determines which words/phrases refer to the same real-world entity across a text (e.g., linking 'she' back to 'Maria').

```
# Using spacy + coreferee (pip install coreferee)
# import coreferee
# nlp.add_pipe('coreferee')
# doc = nlp("Maria went to the store. She bought milk.")
# doc._.coref_chains.print()
```

7. Language Modeling

A language model assigns probabilities to sequences of words — the foundation of text generation, autocomplete, and modern LLMs.

7.1 N-gram Language Models

Predicts the next word using the previous N-1 words (Markov assumption).

```
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk import word_tokenize

text = [word_tokenize("I love natural language processing")]
n = 2
train_data, vocab = padded_everygram_pipeline(n, text)

model = MLE(n)
model.fit(train_data, vocab)
print(model.score("language", ["natural"]))   # P(language | natural)
```

7.2 Neural Language Models (RNN / LSTM)

Uses recurrent neural networks to model sequential dependencies without a fixed window.

```
import torch
import torch.nn as nn

class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden_dim=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x):
        emb = self.embedding(x)
        out, _ = self.lstm(emb)
        return self.fc(out)

model = LSTMLanguageModel(vocab_size=5000)
dummy_input = torch.randint(0, 5000, (2, 10))   # batch=2, seq_len=10
logits = model(dummy_input)
print(logits.shape)   # [2, 10, 5000]
```

7.3 The Transformer & Self-Attention

Transformers replaced recurrence with **self-attention**, letting every token attend directly to every other token — enabling parallel training and long-range context.

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights
```

```
Q = torch.rand(1, 4, 8)   # batch, seq_len, d_k
K = torch.rand(1, 4, 8)
V = torch.rand(1, 4, 8)
output, attn_weights = scaled_dot_product_attention(Q, K, V)
print(output.shape, attn_weights.shape)
```

7.4 Pretrained Language Models (BERT, GPT)

BERT is bidirectional and trained with masked-language-modeling — great for understanding tasks. **GPT** is autoregressive (left-to-right) — great for generation.

```
from transformers import pipeline

# Masked word prediction (BERT-style)
fill_mask = pipeline("fill-mask", model="bert-base-uncased")
print(fill_mask("Paris is the capital of [MASK]."))

# Text generation (GPT-style)
generator = pipeline("text-generation", model="gpt2")
print(generator("Natural language processing is", max_length=30, num_return_sequences=1))
```

8. Core NLP Applications

8.1 Text Classification / Sentiment Analysis

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

texts  = ["I love this product", "This is terrible", "Amazing quality", "Worst purchase ever"]
labels = [1, 0, 1, 0]   # 1=positive, 0=negative

X_train, X_test, y_train, y_test = train_test_split(texts, labels, test_size=0.5, random_state=1)

vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

clf = LogisticRegression()
clf.fit(X_train_vec, y_train)
print(clf.predict(X_test_vec))

# Using a pretrained transformer instead:
from transformers import pipeline
sentiment = pipeline("sentiment-analysis")
print(sentiment("I absolutely loved this movie!"))
```

8.2 Machine Translation

```
from transformers import pipeline

translator = pipeline("translation_en_to_fr", model="Helsinki-NLP/opus-mt-en-fr")
print(translator("Natural language processing is fascinating."))
```

8.3 Text Summarization

8.3.1 Extractive Summarization (TextRank)

```
from gensim.summarization import summarize   # older gensim versions
# For newer gensim, use `sumy` or `networkx`-based TextRank implementations

text = "Your long article text goes here. " * 20
print(summarize(text, ratio=0.2))
```

8.3.2 Abstractive Summarization (Transformers)

```
from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
long_text = "Natural language processing (NLP) is a subfield of AI ... " * 5
print(summarizer(long_text, max_length=60, min_length=20, do_sample=False))
```

8.4 Question Answering

```
from transformers import pipeline

qa = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")
```

```
context = "NLP was popularized in the 1950s with the Turing Test."
result = qa(question="When was NLP popularized?", context=context)
print(result['answer'])
```

8.5 Topic Modeling (LDA)

Discovers latent 'topics' — clusters of co-occurring words — across a document collection.

```
from gensim import corpora
from gensim.models import LdaModel

docs = [["nlp", "language", "text"], ["deep", "learning", "neural", "network"],
        ["text", "mining", "nlp", "data"]]

dictionary = corpora.Dictionary(docs)
corpus = [dictionary.doc2bow(doc) for doc in docs]

lda = LdaModel(corpus, num_topics=2, id2word=dictionary, passes=10)
for idx, topic in lda.print_topics(-1):
    print(f"Topic {idx}: {topic}")
```

8.6 Text Generation

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")
output = generator("Once upon a time in the world of AI,", max_length=50)
print(output[0]['generated_text'])
```

8.7 Conversational AI / Chatbots

Modern chatbots combine intent classification, entity extraction, dialogue management, and (increasingly) large pretrained language models for open-domain response generation.

9. Evaluation Metrics

9.1 Classification Metrics

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_true = [1, 0, 1, 1, 0]
y_pred = [1, 0, 0, 1, 0]

print("Accuracy :", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall   :", recall_score(y_true, y_pred))
print("F1 Score :", f1_score(y_true, y_pred))
```

9.2 Perplexity (Language Models)

Measures how well a probability model predicts a sample — lower perplexity = better model.

```
import math

def perplexity(probabilities):
    n = len(probabilities)
    log_sum = sum(math.log(p) for p in probabilities)
    return math.exp(-log_sum / n)

probs = [0.4, 0.2, 0.6, 0.8]   # P(actual next word) at each step
print(perplexity(probs))
```

9.3 BLEU Score (Machine Translation)

```
from nltk.translate.bleu_score import sentence_bleu

reference = [["the", "cat", "sat", "on", "the", "mat"]]
candidate = ["the", "cat", "is", "on", "the", "mat"]
print(sentence_bleu(reference, candidate))
```

9.4 ROUGE Score (Summarization)

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rougeL'], use_stemmer=True)
scores = scorer.score(
    "the quick brown fox jumps over the lazy dog",
    "a quick brown fox jumped over a lazy dog"
)
print(scores)
```

10. End-to-End Mini Project: Sentiment Analysis

A compact pipeline combining preprocessing, TF-IDF representation, and a classifier.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# 1. Sample data
data = [
    ("I absolutely loved this movie, fantastic acting!", 1),
    ("Worst film I have ever seen, total waste of time.", 0),
    ("A masterpiece, brilliant direction and story.", 1),
    ("Terrible plot and boring characters.", 0),
    ("One of the best movies this year, highly recommend!", 1),
    ("I hated every minute, would not watch again.", 0),
]
texts, labels = zip(*data)

# 2. Preprocessing
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = re.sub(r"[^a-zA-Z\s]", "", text.lower())
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words]
    return " ".join(tokens)

clean_texts = [preprocess(t) for t in texts]

# 3. Feature extraction
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(clean_texts)

# 4. Train / test split
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.34, random_state=42)

# 5. Train model
clf = LogisticRegression()
clf.fit(X_train, y_train)

# 6. Evaluate
preds = clf.predict(X_test)
print(classification_report(y_test, preds))

# 7. Predict on new text
new_review = preprocess("What an incredible, heartwarming film!")
vec = vectorizer.transform([new_review])
print("Prediction:", clf.predict(vec))   # 1 = positive
```

11. Summary Roadmap

A condensed map of the full concept hierarchy covered in this guide.

Stage	Concepts	Key Tools
Preprocessing	Tokenization, Stopwords, Stemming, Lemmatization, Normalization	nltk, spaCy, re
Representation	BoW, TF-IDF, N-grams, Word2Vec, GloVe, FastText, BERT embeddings	sklearn, gensim, transformers
Syntax	POS tagging, Constituency & Dependency parsing, Chunking	nltk, spaCy
Semantics	NER, WSD, SRL, Coreference resolution	spaCy, nltk, allennlp
Language Models	N-gram LM, RNN/LSTM, Transformers, BERT/GPT	nltk, PyTorch, transformers
Applications	Classification, Translation, Summarization, QA, Topic Modeling	sklearn, transformers, gensim
Evaluation	Accuracy/F1, Perplexity, BLEU, ROUGE	sklearn, nltk, rouge-score

Note: This guide is a practical companion, not an exhaustive academic reference. For deeper theory, pair it with resources like Jurafsky & Martin's *Speech and Language Processing* and the Hugging Face Transformers documentation.